

mongoDB[®]

document database
performance
ease data access
no-schema

DB Ranking

#5 [Overall](#)

#1 [Document stores](#)

<https://db-engines.com/en/system/MongoDB>

Initial release: 2009

Stable release: 6.0.5 / 2023-03-13

repository: github.com/mongodb/mongo

414 systems in ranking, April 2023

Rank			DBMS	Database Model	Score		
Apr 2023	Mar 2023	Apr 2022			Apr 2023	Mar 2023	Apr 2022
1.	1.	1.	Oracle +	Relational, Multi-model ⓘ	1228.28	-33.01	-26.54
2.	2.	2.	MySQL +	Relational, Multi-model ⓘ	1157.78	-25.00	-46.38
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model ⓘ	918.52	-3.49	-19.94
4.	4.	4.	PostgreSQL +	Relational, Multi-model ⓘ	608.41	-5.41	-6.05
5.	5.	5.	MongoDB +	Document, Multi-model ⓘ	441.90	-16.89	-41.48
6.	6.	6.	Redis +	Key-value, Multi-model ⓘ	173.55	+1.10	-4.05
7.	7.	↑ 8.	IBM Db2	Relational, Multi-model ⓘ	145.49	+2.57	-14.97
8.	8.	↓ 7.	Elasticsearch	Search engine, Multi-model ⓘ	141.08	+2.01	-19.76
9.	9.	↑ 10.	SQLite +	Relational	134.54	+0.72	+1.75
10.	10.	↓ 9.	Microsoft Access	Relational	131.37	-0.69	-11.41

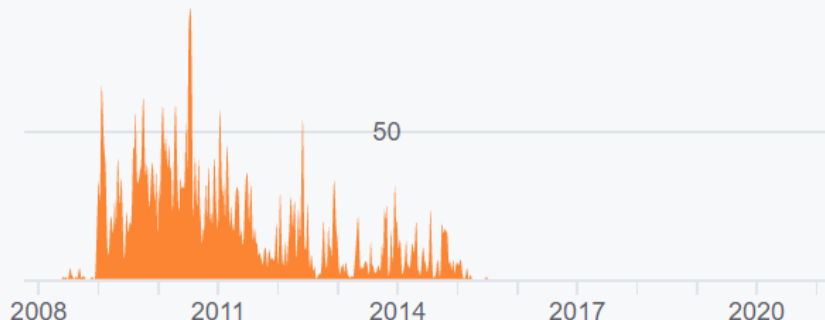
los creadores de mongodb



erh

#1

6,179 commits 981,350 ++ 566,247 --



Eliot Horowitz

<https://github.com/erh>

[@eliothorowitz](https://twitter.com/eliothorowitz)

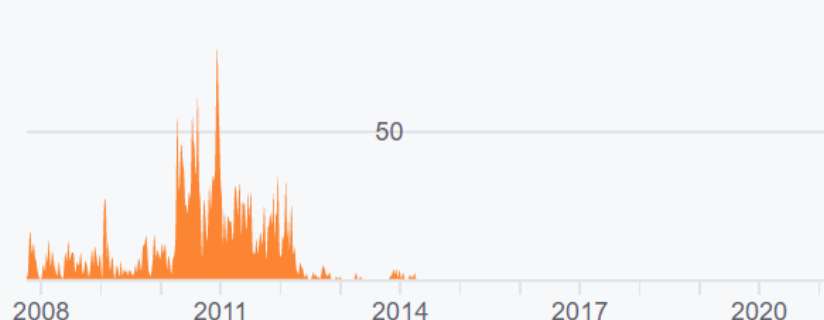
https://en.wikipedia.org/wiki/Eliot_Horowitz



dwight

#2

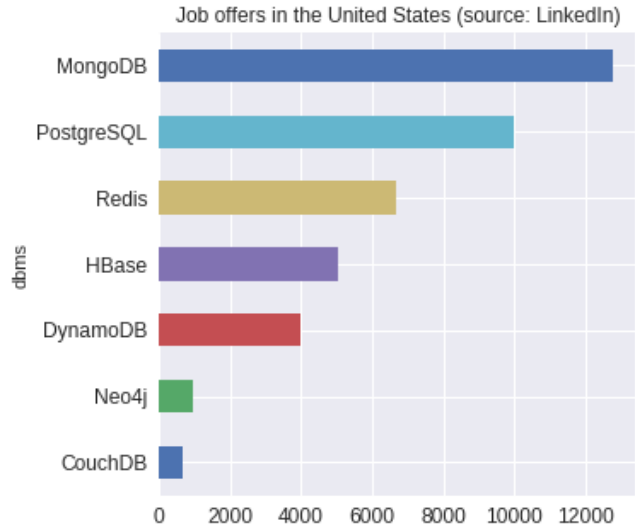
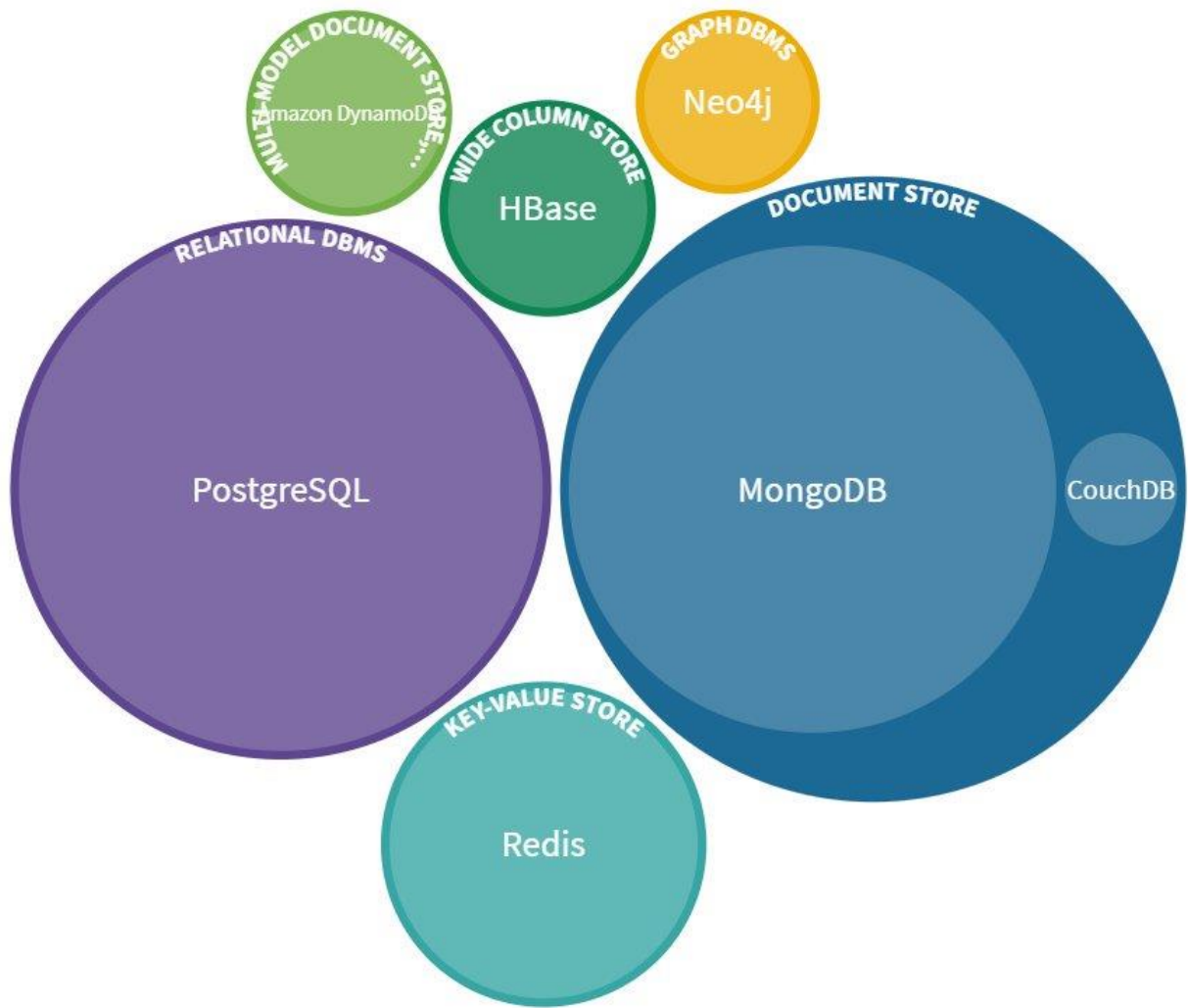
3,477 commits 876,158 ++ 793,237 --



Dwight Merriman

<https://github.com/dwight>

https://en.wikipedia.org/wiki/Dwight_Merriman



MongoDB is a JSON document database (though technically data is stored in a binary form of JSON known as BSON)

```
{
  "_id" : ObjectId("4d0b6da3bb30773266f39fea"),
  "country" : {
    "$ref" : "countries",
    "$id" : ObjectId("4d0e6074deb8995216a8309e")
  },
  "famous_for" : [
    "beer",
    "food"
  ],
  "last_census" : "Sun Jan 07 2018 00:00:00 GMT -0700 (PDT)",
  "mayor" : {
    "name" : "Ted Wheeler",
    "party" : "D"
  },
  "name" : "Portland",
  "population" : 582000,
  "state" : "OR"
}
```



JSON document

Ejemplo de un tuit en formato JSON

<https://gist.github.com/aaizemberg/c297e70f46a57affee76d8886a607298>

```
{'contributors': None,
'coordinates': None,
'created_at': 'Sun Sep 01 19:05:54 +0000 2019',
'display_text_range': [0, 92],
'entities': {'hashtags': [],
'media': [{'display_url': 'pic.twitter.com/OjmCRfs0Vv',
'expanded_url': 'https://twitter.com/aaizemberg/status/1168238568688553989/photo/1',
'id': 1168238539403943944,
'id_str': '1168238539403943944',
'indices': [93, 116],
'media_url': 'http://pbs.twimg.com/media/EDZqwxIX4AgqZkz.jpg',
'media_url_https': 'https://pbs.twimg.com/media/EDZqwxIX4AgqZkz.jpg',
'sizes': {'large': {'h': 2048, 'resize': 'fit', 'w': 1153},
'medium': {'h': 1200, 'resize': 'fit', 'w': 676},
'small': {'h': 680, 'resize': 'fit', 'w': 383},
'thumb': {'h': 150, 'resize': 'crop', 'w': 150}}},
```

"_id" → ObjectId o generado por el usuario

```
> db.egresados.findOne()
```

```
{  
  "_id" : ObjectId("5d712e759bec0f1238869a1a"),  
  "Legajo" : "53213",  
  "Apellido" : "Freddo",  
  "Nombre" : "Ramiro",  
  "Título" : "Ingeniero en Informática",  
  "promedio_lineal" : 6.68  
}
```

```
> db.egresados.findOne()._id  
ObjectId("5d712e759bec0f1238869a1a")
```

```
> db.egresados.findOne()._id.getTimestamp()  
ISODate("2019-09-05T15:49:09Z")
```

```
> db.egresados.findOne()._id.toString()  
ObjectId("5d712e759bec0f1238869a1a")
```

```
> db.egresados.findOne()._id.valueOf()  
5d712e759bec0f1238869a1a
```

"_id" → ObjectId o generado por el usuario

```
> db.testing.insertOne({"_id": 1, "value": 10})  
{ "acknowledged" : true, "insertedId" : 1 }
```

```
> db.testing.insertOne({"_id": "2", "value": 20})  
{ "acknowledged" : true, "insertedId" : "2" }
```

```
> db.testing.insertOne({"value": 30})  
{ "acknowledged" : true, "insertedId" : ObjectId("5e95cbbf8d72de7f0b97e4bd") }
```

```
> db.testing.find()  
{ "_id" : 1, "value" : 10 }  
{ "_id" : "2", "value" : 20 }  
{ "_id" : ObjectId("5e95cbbf8d72de7f0b97e4bd"), "value" : 30 }
```

levantando el servicio y entrando al shell (mongosh)

prender y apagar el servicio

#

\$ sudo service mongod start

\$ sudo service mongod stop

levantar el shell

#

\$ mongosh --host localhost --port 27017 # es equivalente a

\$ mongosh

esta otra instrucción

MongoDB Shell, mongosh

The MongoDB Shell ([monaosh](#)) is not installed with MongoDB Server. You need to follow the [monaosh installation instructions](#) to download and install [mongosh](#) separately.

> show dbs

mostrar las bases de datos

> help

db.help()	help on db methods
db.mycoll.help()	help on collection methods
sh.help()	sharding helpers
rs.help()	replica set helpers
help admin	administrative help
help connect	connecting to a db help
help keys	key shortcuts
help misc	misc things to know
help mr	mapreduce
show dbs	show database names
show collections	show collections in current database
show users	show users in current database
show profile	show most recent system.profile entries with time >= 1ms
show logs	show the accessible logger names
show log [name]	prints out the last segment of log in memory, 'global' is default
use <db_name>	set current database
db.mycoll.find()	list objects in collection mycoll
db.mycoll.find({ a : 1 })	list objects in mycoll where a == 1
it	result of the last line evaluated; use to further iterate
DBQuery.shellBatchSize = x	set default number of items to display on shell
exit	quit the mongo shell

insertando un registro

```
# usar una base de datos
```

```
#
```

```
use test
```

```
# crear una colección (o tabla)
```

```
#
```

```
db.createCollection("usuarios")
```

```
# insertando 1 registro
```

```
#
```

```
db.usuarios.insert({"nombre": "Ariel", "apellido": "Aizemberg", "username":  
"aaizemberg"})
```

colecciones o tablas?

probar estas 3 instrucciones

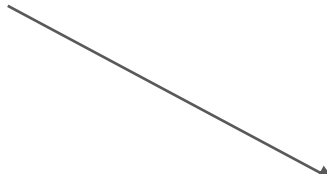
1. > show collections
2. > show tables
3. > db.getCollectionNames()

> show dbs

// para listar las bases de datos

Importando un archivo .tsv (mongoimport)

```
$ mongoimport --headerline --db test --collection internos --type tsv <
internos.tsv
```



```
connected to: 127.0.0.1
```

```
2018-10-03T15:40:31.471-0300 check 9 290
```

```
2018-10-03T15:40:31.472-0300 imported 289 objects
```

```
> db.internos.count()
```

```
> db.internos.countDocuments()
```

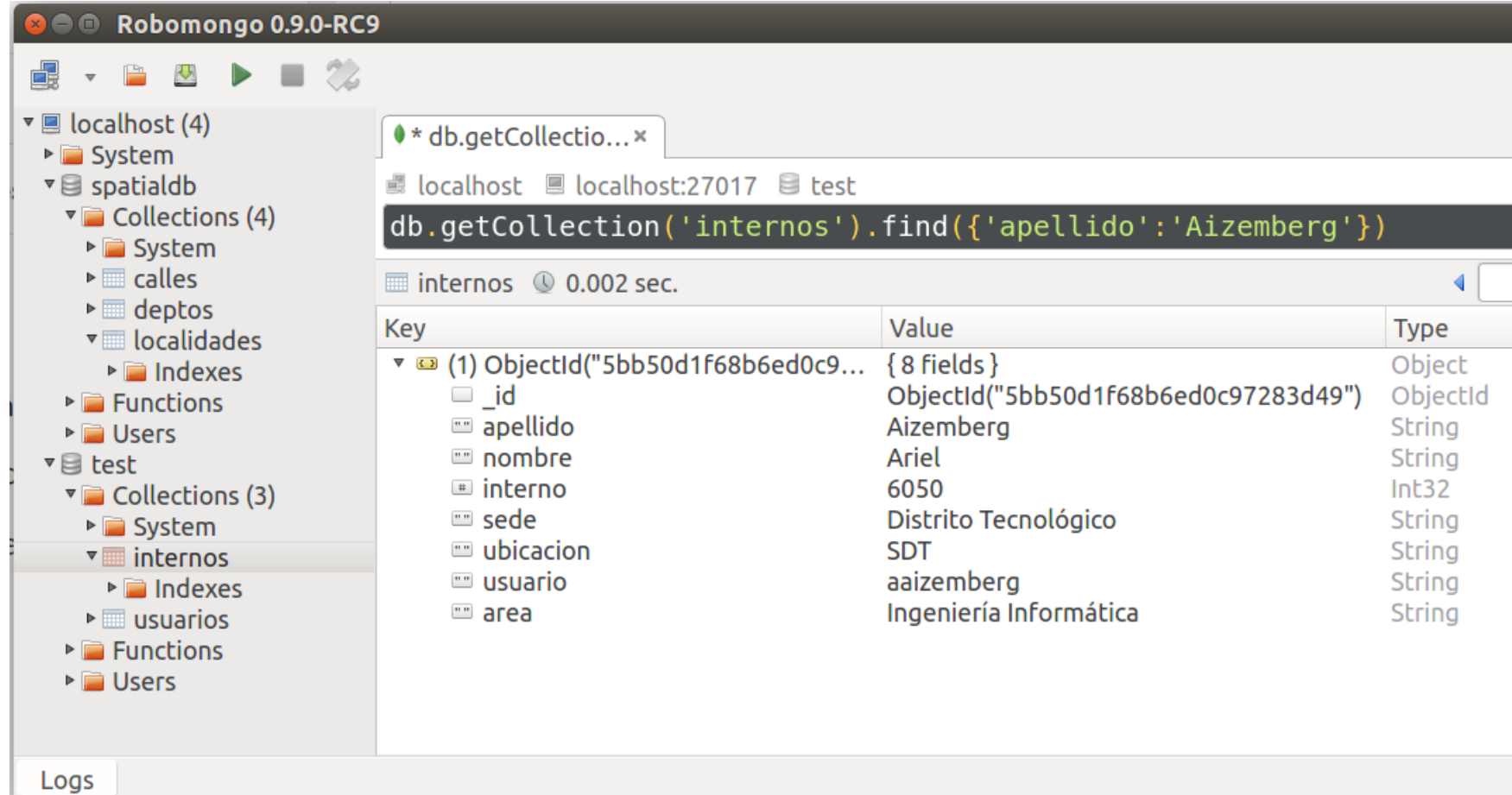
```
289
```

Mostrando el primer registro (pretty print)

```
> db.internos.findOne()
```

```
{  
  "_id" : ObjectId("5f4e8167587f8b5c6d24ec31"),  
  "nombre" : "Aguiar Andrea",  
  "user" : "aaguiar",  
  "int" : 4741,  
  "area" : "Calidad Educativa",  
  "sector" : "Calidad Educativa",  
  "sede" : "Central"  
}
```

Robomongo (GUI) → Robo 3T (Studio \$)



The screenshot displays the Robomongo 0.9.0-RC9 application window. On the left, a sidebar shows the database structure for 'localhost (4)', including 'spatialdb' and 'test' databases. The 'test' database is selected, showing collections like 'internos'. The main area shows a query executed in the 'internos' collection: `db.getCollection('internos').find({'apellido': 'Aizemberg'})`. The results are displayed in a table with columns 'Key', 'Value', and 'Type'.

Query: `db.getCollection('internos').find({'apellido': 'Aizemberg'})`

Execution time: 0.002 sec.

Key	Value	Type
(1) ObjectId("5bb50d1f68b6ed0c9...")	{ 8 fields }	Object
_id	ObjectId("5bb50d1f68b6ed0c97283d49")	ObjectId
apellido	Aizemberg	String
nombre	Ariel	String
interno	6050	Int32
sede	Distrito Tecnológico	String
ubicacion	SDT	String
usuario	aaizemberg	String
area	Ingeniería Informática	String

Logs

MongoDB Compass

Activities

MongoDB Compass

mar 14:48

MongoDB Compass - Connect

ConnectViewCollectionHelp

CREATE FREE ATLAS CLUSTER
Includes 512 MB of data storage.
[Learn more](#)

New Connection

My Cluster

4 DBS5 COLLECTIONS

filter

7 dbs

- internos
- narda
- neighborhoods
- restaurants
- admin
- config
- local

Favorites

JAN 29, 2019 5:08 PM
M001 Sandbox

JAN 29, 2019 5:25 PM
M001 RS

RECENTS

MAR 14, 2019 2:25 PM
localhost:27017

ConnectViewCollectionHelp

localhost:27017STANDALONE

MongoDB 4.0.8 Community

7dbs.narda

DOCUMENTS104TOTAL SIZE24.8KB AVG. SIZE244B

INDEXES1TOTAL SIZE16.0KB AVG. SIZE16.0KB

DocumentsAggregationsSchemaExplain PlanIndexesValidation

FILTEROPTIONSANALYZERESET...

Query returned 104 documents. This report is based on a sample of 104 documents (100.00%).

date

string

Sun Jul 22 14:44:14 +0000 2018Tue Jul 24 23:19:13 +0000 2018

Sun Jul 22 20:41:48 +0000 2018Tue Jul 24 02:20:17 +0000 2018

Tue Jul 24 19:14:02 +0000 2018Tue Jul 24 23:12:16 +0000 2018

Mon Jul 23 19:52:42 +0000 2018Sun Jul 22 15:02:01 +0000 2018

fav

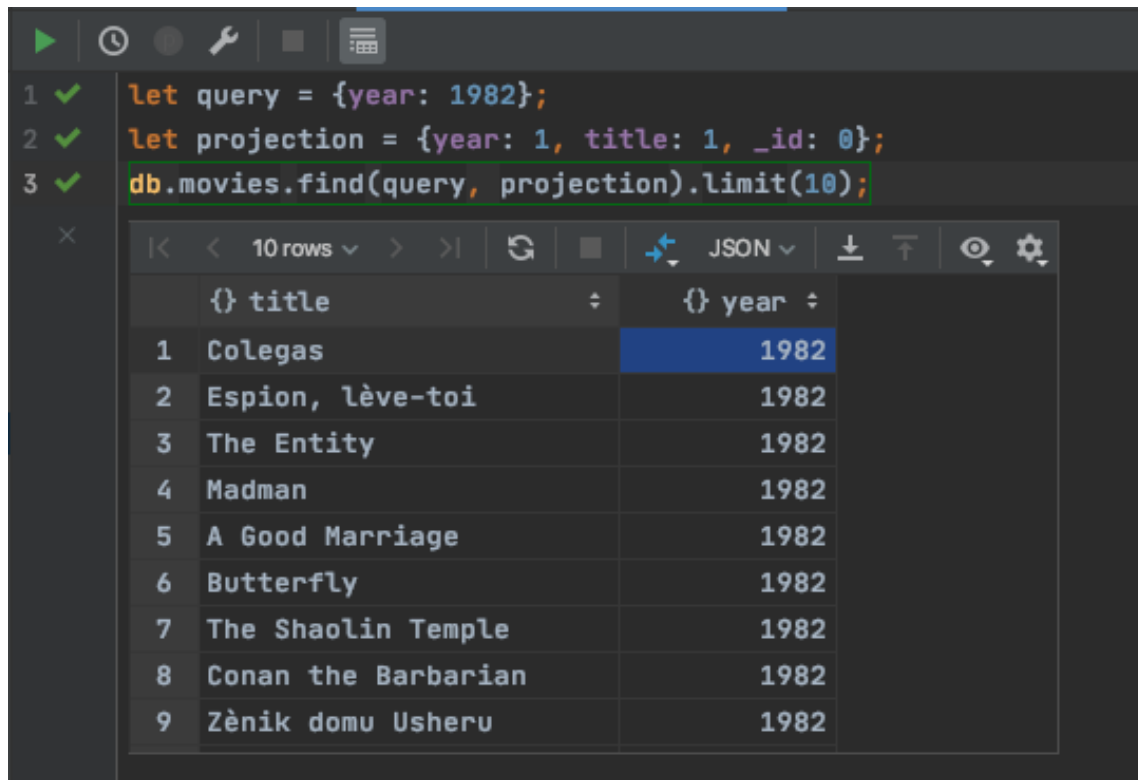
int32

44%22%0%min: 553max:

CONNECT

DataGrip

a great database IDE to work with MongoDB Atlas



The screenshot displays the DataGrip IDE interface. On the left, a code editor shows a MongoDB query with three lines of JavaScript code, each preceded by a green checkmark icon:

```
1 let query = {year: 1982};  
2 let projection = {year: 1, title: 1, _id: 0};  
3 db.movies.find(query, projection).limit(10);
```

Below the code editor, a table view displays the results of the query. The table has two columns: 'title' and 'year'. The first row is highlighted in blue. The table contains 10 rows of data, all with the year 1982.

	{ } title	{ } year
1	Colegas	1982
2	Espion, lève-toi	1982
3	The Entity	1982
4	Madman	1982
5	A Good Marriage	1982
6	Butterfly	1982
7	The Shaolin Temple	1982
8	Conan the Barbarian	1982
9	Zènik domu Usheru	1982

NoSQLBooster for MongoDB - MongoBooster

The screenshot displays the NoSQLBooster for MongoDB application interface. On the left, the 'Connection Tree' panel shows a hierarchy of database connections. The 'test' database is selected, and the 'unicorns' collection is highlighted. Below the tree, the 'My Queries' and 'Samples' tabs are visible. The 'Samples' tab is active, showing a list of sample queries. The '01.intro.js' file is selected, which contains the following code:

```
var db = mongo.IDatabase
DBQuery var
findById find
show dbs command
use <db> command
findByObjectId find
show databases command
findWhereObjectIdFromDate find
```

The main editor area on the right shows the code for the selected file. The 'test:unicorns' tab is active, and the 'localhost_40017' connection is selected. The 'test' database is also selected. The code editor shows the same code as the 'Samples' tab, with a tooltip for the 'var db: mongo.IDatabase' line.

CRUD and Nesting

use DATABASE_NAME

crea o usa una DB

show dbs

muestra las

db

te

informa en que DB estas

show collections

db.towns.insert(<JSON>)

inserta un registro

db.town.findOne()

busca 1 registro

db.town.find()

Se puede ver cómo está implementada una función, si la llamamos sin los parentesis, ej: db.town.findOne

buscando con expresiones regulares y rangos

```
> db.towns.find( { name : /^P/ }, { _id: 0, name : 1, population : 1 } )  
{ "name" : "Punxsutawney", "population" : 6200 }  
{ "name" : "Portland", "population" : 582000 }
```

```
> db.towns.find( { name : /^P/ , population : { $lt : 10000 } }, {  
_id: 0, name : 1, population : 1 } )  
{ "name" : "Punxsutawney", "population" : 6200 }
```

```
> var pop_range = { $lt: 1000000, $gt: 10000 }  
> db.towns.find( { name : /^P/ , population : pop_range}, { _id: 0,  
name : 1, population : 1 } )
```

```
> db.internos.find({nombre: /aiz.*/i})
```

\$all & \$nin

```
> db.towns.find( { famousFor : { $all : [ 'food' , 'beer' ] } }, { _id : 0,  
name:1, famousFor:1 } )
```

```
> db.towns.find( { famousFor : { $nin : [ 'food' , 'beer' ] } }, { _id : 0,  
name:1, famousFor:1 } )
```

\$nin selects the documents where:

- the field value is not in the specified array or
- the field does not exist. **# Be careful, do not misspell the field. Remember that the fields are case sensitive.**

The records do not necessarily have the same structure

```
db.countries.insert({_id : "us" ,name : "United States",  
  exports : {foods : [{ name : "bacon" , tasty : true }, { name :  
"burgers" } ]}})
```

```
db.countries.insert({_id : "ca" ,name : "Canada" ,  
  exports : {foods : [{ name : "bacon" , tasty : false }, { name :  
"syrup" , tasty : true } ]}})
```

```
db.countries.insert({_id : "mx" ,name : "Mexico" ,  
  exports : {foods : [{ name : "salsa" , tasty : true , condiment  
: true } ]}})
```

operadores (**and** implícito y **or**)

```
> db.countries.find( { _id : "mx", name : "United States" }, {  
_id:1, name:1 } )  
# no retorna nada
```

```
> db.countries.find( { $or : [ { _id : "mx" }, { name : "United  
States" } ] }, { _id:1, name:1 } )  
{ "_id" : "us", "name" : "United States" }  
{ "_id" : "mx", "name" : "Mexico" }
```

update

```
> db.towns.update( { name : "Portland" }, { $set : { "state" :  
"OR" } } )
```

```
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1  
})
```

```
> db.towns.find({}, {"_id":0, name:1, state:1})  
{ "name" : "New York" }  
{ "name" : "Punxsutawney" }  
{ "name" : "Portland", "state" : "OR" }
```

updateOne (desde python)

```
import pymongo

from pymongo import MongoClient

client = MongoClient('mongodb://<usuario>:<password>@<host>:<port>/' 'elecciones-2019')
db = client['elecciones-2019']
news_collection = db['news']

aa_key = '*****'
cursor = db.news.find({"engagement": None})
for entry in cursor:
    id = entry['_id']
    link = entry['link']
    engagement = sharedcount_fb_engagement( link )
    myquery = { "_id": id }
    newvalues = { "$set": { "engagement": engagement } }
    news_collection.update_one(myquery, newvalues)
    print('{0} {1}'.format(link, engagement))
```

delete

```
> db.countries.count()
```

```
3
```

```
> var badBacon = { 'exports.foods' : { $elemMatch : { name :  
'bacon', tasty : false } } }
```

```
> db.countries.find(badBacon)
```

```
{ "_id" : "ca", "name" : "Canada", "exports" : { "foods" : [ {  
"name" : "bacon", "tasty" : false }, { "name" : "syrup", "tasty" :  
true } ] } }
```

```
> db.countries.deleteOne(badBacon)
```

```
> db.countries.count()
```

```
2
```

indexing

default `_id` index

MongoDB creates a unique index on the `_id` field during the creation of a collection. The `_id` index prevents clients from inserting two documents with the same value for the `_id` field. You cannot drop this index on the `_id` field.

By default, all collections have an index on the `_id` field.

```
// Show all indexes of the current database
db.getCollectionNames().forEach(function(collection) {
    print("Indexes for the " + collection + " collection:");
    printjson(db[collection].getIndexes());
});
```

single field indexes

```
> db.phones.count()
```

```
100,000 rows
```

```
> db.phones.find({display: "+1 800-5650001"}).explain()      → 61 ms
```

```
> db.phones.ensureIndex( { display : 1 }, { unique : true } )
```

```
> db.phones.find({display: "+1 800-5650001"}).explain()      → 0 ms
```

```
// index on nested values
```

```
> db.phones.ensureIndex( { "components.area": 1 }, { background : 1 })
```

+info: <https://docs.mongodb.com/manual/core/index-single/>

Command	Description
mongodump	Exports data from Mongo into .bson files. That can mean entire collections or databases, filtered results based on a supplied query, and more.
mongofiles	Manipulates large GridFS data files (GridFS is a specification for BSON files exceeding 16 MB).
mongooplog	Polls operation logs from MongoDB replication operations.
mongorestore	Restores MongoDB databases and collections from backups created using mongodump.
mongostat	Displays basic MongoDB server stats.
mongoexport	Exports data from Mongo into CSV (comma-separated value) and JSON files. As with mongodump, that can mean entire databases and collections or just some data chosen on the basis of query parameters.
mongoimport	Imports data into Mongo from JSON, CSV, or TSV (term-separated value) files. We'll use this tool on Day 3.
mongoperf	Performs user-defined performance tests against a MongoDB server.
mongos	Short for “MongoDB shard,” this tool provides a service for properly routing data into a sharded MongoDB cluster (which we will not cover in this chapter).
mongotop	Displays usage stats for each collection stored in a Mongo database.
bsondump	Converts BSON files into other formats, such as JSON.

Aggregated Queries

Collection

```
db.orders.aggregate( [  
  $match stage → { $match: { status: "A" } },  
  $group stage → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }  
] )
```

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }

orders

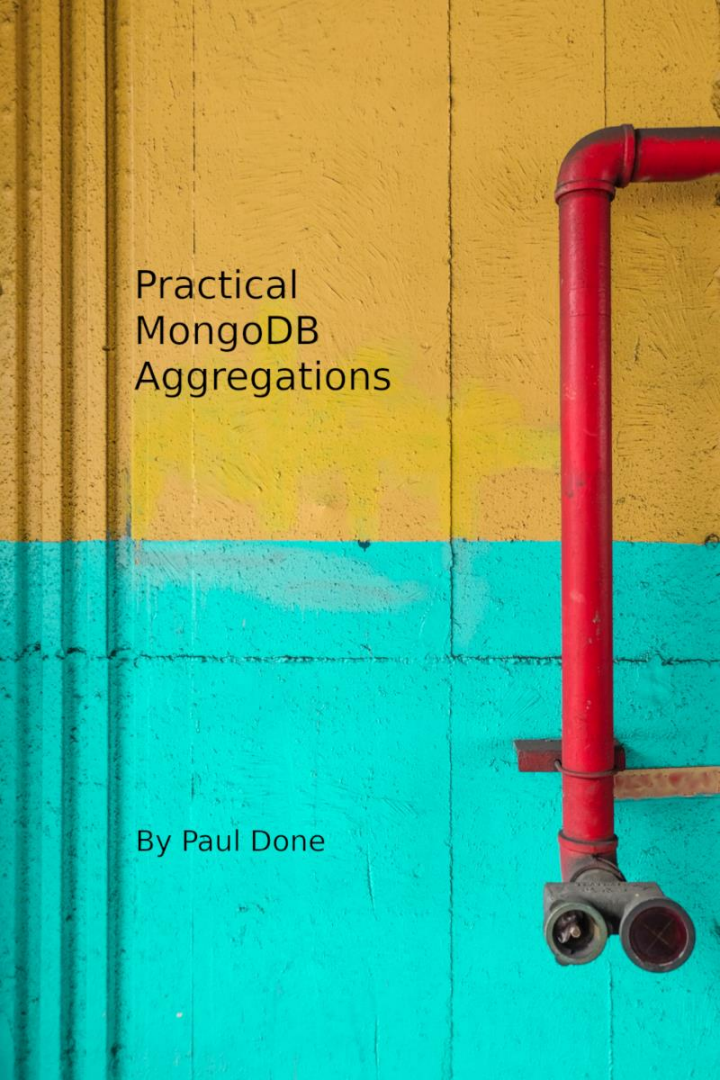
\$match

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }

\$group

Results	
{	<code>_id: "A123",</code> total: 750
}	

{	<code>_id: "B212",</code> total: 200
}	

A red fire hose is positioned vertically on the left side of the image. It has a 90-degree elbow at the top and a nozzle at the bottom. The hose is set against a background of two textured panels: a yellow one on top and a blue one on the bottom. The text 'Practical MongoDB Aggregations' is written in black on the yellow panel, and 'By Paul Done' is written in black on the blue panel.

Practical MongoDB Aggregations

By Paul Done

<https://www.practical-mongodb-aggregations.com/>

Aggregated Queries

```
SELECT tipo, Count(*)  
FROM hospitales  
GROUP BY tipo  
ORDER BY Count(*) DESC
```

```
db.hospitales.aggregate([  
  { $group : { _id: "$properties.TIPO", count:{$sum:1} } },  
  { $sort : { count : -1 } }  
])
```

```
{ "_id" : "Hospital especializado", "count" : 20 }  
{ "_id" : "Hospital de agudos", "count" : 13 }  
{ "_id" : "Hospital de niños", "count" : 3 }
```

Aggregated Queries

```
db.phones.count({ 'components.number': { $gt : 5599999 } } )
```

```
db.phones.distinct('components.number',{ 'components.number': { $lt :  
5550005 } } )
```

```
db.cities.aggregate([ { $match: { 'timezone': { $eq: 'Europe/London' } } },  
{ $group: { _id: 'averagePopulation', avgPop: { $avg: '$population' } } }  
])
```

```
db.cities.aggregate([  
  { $match: { 'timezone': { $eq: 'Europe/London' } } },  
  { $sort: { population: -1 } },  
  { $project: { _id: 0, name: 1, population: 1 } }  
])
```

MapReduce

In MongoDB, the map step involves creating a mapper function that calls an ***emit()*** function. The benefit of this approach is you can emit more than once per document. The ***reduce()*** function accepts a single key and a list of values that were emitted to that key. Finally, Mongo provides an optional third step called ***finalize()***, which is executed only once per mapped value after the reducers are run. This allows you to perform any final calculations or cleanup you may need.

[2004] [MapReduce: Simplified Data Processing on Large Clusters](https://en.wikipedia.org/wiki/MapReduce)

<https://en.wikipedia.org/wiki/MapReduce>

MapReduce

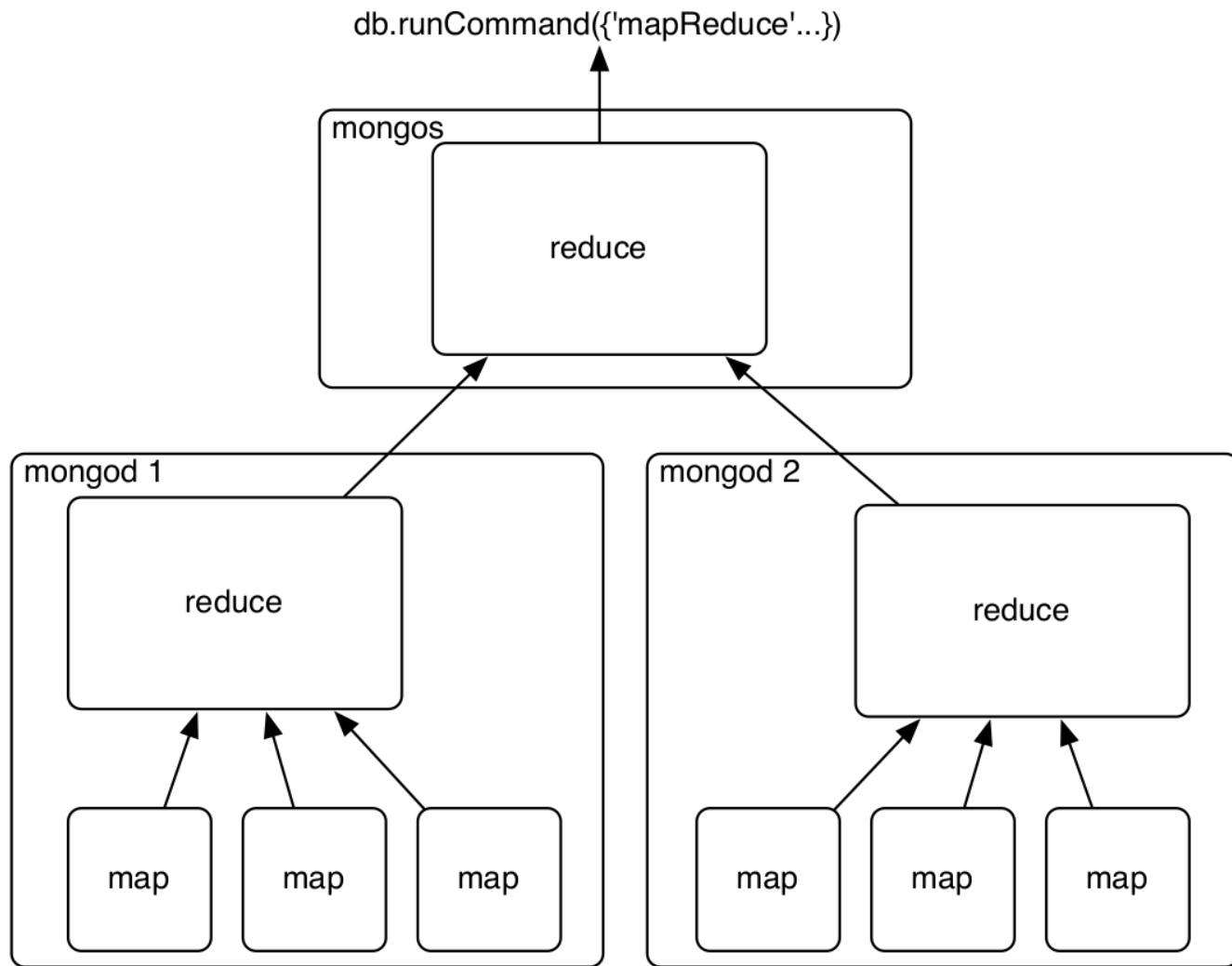
ejemplo de la página 120

1. <https://media.pragprog.com/titles/pwrdata/code/mongo/distinctDigits.js>
2. <https://media.pragprog.com/titles/pwrdata/code/mongo/map1.js>
3. <https://media.pragprog.com/titles/pwrdata/code/mongo/reduce1.js>

```
results = db.runCommand({  
  mapReduce: 'phones',  
  map: map,  
  reduce: reduce,  
  out: 'phones.report' })
```



```
{ "result" : "phones.report",  
  "timeMillis" : 2464,  
  "counts" : {  
    "input" : 100000,  
    "emit" : 100000,  
    "reduce" : 25719,  
    "output" : 3479  
  },  
  "ok" : 1 }
```



user-defined functions in system.js

any JavaScript function can be stored in a special collection named **system.js**

```
> db.system.js.save({  
  _id: 'getLast',  
  value: function(collection) {  
    return collection.find({}).sort({'_id':-1}).limit(1)[0]; }  
})
```

```
> use book  
> db.loadServerScripts()  
> getLast(db.phones).display
```

+8 800-5649989

accessing mongoDB from python

```
$ pip install pymongo
```

```
import pymongo
from pymongo import MongoClient
client = MongoClient()          # client = MongoClient('localhost', 27017)
                                # client =
MongoClient('mongodb://localhost:27017/')
```

```
db = client['book']
collection = db.countries
for c in collection.find({}, {'name':1}):
    print c['name']
```

United States

Canada

Mexico

Replica sets



Replica Sets (rs)

Mongo's default port is 27017, so we'll start up each server on other ports.

```
$ mkdir ./mongo1 ./mongo2 ./mongo3
$ mongod --replSet book --dbpath ./mongo1 --port 27011
$ mongod --replSet book --dbpath ./mongo2 --port 27012
$ mongod --replSet book --dbpath ./mongo3 --port 27013

$ mongo localhost:27011
> rs.initiate({ _id: 'book', members: [
  {_id: 1, host: 'localhost:27011'},
  {_id: 2, host: 'localhost:27012'},
  {_id: 3, host: 'localhost:27013'} ] })
> rs.status().ok
```

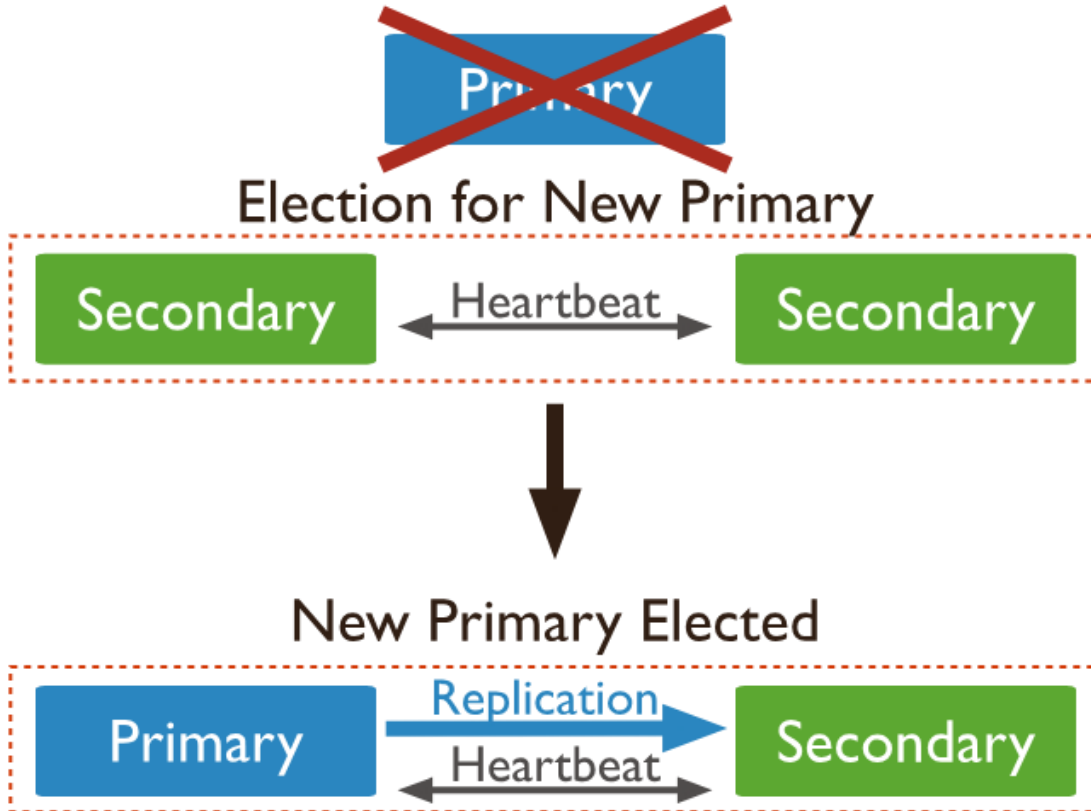
Viewing the replica set from Robo3T

The screenshot shows the Robo3T - 1.2 application window. On the left, a tree view displays the database structure. The 'test' database is expanded, showing a 'ReplicaSet (3)' with three nodes: 'localhost:27011 [Secondary]', 'localhost:27012 [Primary]' (selected), and 'localhost:27013 [Secondary]'. Below this, the 'echo' collection is expanded, showing its 'Indexes (1)'. The main panel on the right shows a query editor with the command `db.getCollection('echo').find({})`. Below the editor, a toolbar indicates the query is for the 'echo' collection, took '0 sec.', and shows pagination controls for 0 to 50 results. The results are displayed in a table with three columns: 'Key', 'Value', and 'Type'.

Key	Value	Type
(1) ObjectId("5bbac61d40f45d...")	{ 2 fields }	Object
_id	ObjectId("5bbac61d40f45d38d44367...")	ObjectId
say	HELLO!	String
(2) ObjectId("5bbacb52486a3...")	{ 2 fields }	Object
(3) ObjectId("5bbaccd16c339f...")	{ 2 fields }	Object

At the bottom left, there is a 'Logs' button.

Replica Set Elections



Sharding

One of the core goals of Mongo is to provide safe and quick handling of very large datasets. The clearest method of achieving this is through horizontal sharding by value ranges - or just sharding for brevity.

mongoDB: allows sharding from version 1.6

<https://docs.mongodb.com/manual/sharding/>

- Seguir el ejemplo, de la pag. 127 o mirar [este video "Sharding a MongoDB Collection"](#)
- De donde viene el termino SHARD → [https://en.wikipedia.org/wiki/Shard_\(database_architecture\)](https://en.wikipedia.org/wiki/Shard_(database_architecture))

more info

Documentación sitio oficial MongoDB -

<https://www.mongodb.com/docs/manual/reference/>

